

Infrastructure as Code: Terraform



Disusun Oleh :

Kanjeng Dhimas Cahyoherlina

BAB I

PENDAHULUAN

1.1 Tujuan Penugasan

Penugasan ini bertujuan untuk merancang, mengimplementasikan, serta memverifikasi infrastruktur terautomasi menggunakan pendekatan *Infrastructure as Code* (IaC) dengan Terraform serta manajemen konfigurasi terpusat menggunakan Ansible.

Berbeda dengan lingkungan *Production* yang mengutamakan isolasi jaringan fisik melalui *Private Subnet* dan *NAT Gateway*, implementasi pada penugasan ini difokuskan pada lingkungan *Development* (Dev). Oleh karena itu, pendekatan yang digunakan adalah *Cost Optimization* (Efisiensi Biaya). Arsitektur disesuaikan untuk menekan biaya operasional AWS seminimal mungkin tanpa mengorbankan logika otomatisasi dan keamanan (menggunakan isolasi logis). Penugasan ini mencakup 4 (empat) pilar utama:

1. **Otomatisasi Provisioning Infrastruktur AWS dengan Advanced Terraform:** Membangun jaringan *Virtual Private Cloud* (VPC) AWS secara otomatis menggunakan modul Terraform tingkat lanjut yang mencakup *Remote Backend* (S3 & DynamoDB State Locking), *Dynamic Block*, *Conditional Expression*, serta fungsi pengolahan subnet dinamis (cidrsubnet).
2. **Topologi Jaringan Cost-Optimized (Tanpa NAT Gateway):** Mengatur infrastruktur jaringan yang ramah anggaran dengan meniadakan NAT Gateway dan Elastic IP (yang memakan biaya per jam). Seluruh resource (termasuk Database Nodes) diletakkan pada *Public Subnet* secara terpusat dan mengandalkan *Internet Gateway* (IGW) murni untuk akses keluar (*outbound*).
3. **Isolasi Logis Node MySQL Terpusat:** Mengalokasikan *Managed Nodes* (EC2 Instance) berbasis Ubuntu Linux di *Public Subnet* yang disiapkan khusus untuk kluster basis data MySQL. Meskipun berada di subnet publik, instans ini diamankan secara ketat (isolasi logis) menggunakan *Security Group* dinamis yang hanya menerima lalu lintas masuk (*inbound*) dari Bastion Host, sehingga mustahil diakses langsung dari internet umum.
4. **Integrasi Otomatisasi Ansible Control Node pada EC2 Bastion Host:** Mengonfigurasi EC2 Bastion Host sekaligus berfungsi sebagai Ansible Control Node yang mengontrol *Managed Nodes* (Database Nodes) di AWS secara langsung melalui jaringan privat VPC (10.0.1.x), mensimulasikan praktik manajemen konfigurasi terpusat secara efisien tanpa memerlukan *Virtual Machine* lokal.

1.2 Perencanaan Infrastruktur & Jaringan AWS

Pada standar industri (*Best Practice Production*), server basis data diwajibkan berada di dalam *Private Subnet* dan mengakses internet keluar menggunakan NAT Gateway. Namun, untuk kebutuhan praktikum dan *Development*, keberadaan NAT Gateway akan menimbulkan tagihan AWS yang berjalan terus-menerus.

Sebagai solusi cerdas manajemen biaya, seluruh instans diletakkan di *Public Subnet*. Keamanan tetap terjamin karena kita memindahkan lapisan proteksi dari level *Network/Subnet* (Routing) ke level *Firewall* (*Security Group*). Pendekatan ini meminimalkan risiko serangan (*attack surface*) dari luar tanpa membebani anggaran belanja cloud.

A. Perencanaan Network VPC & Subnetting

Komponen Jaringan	Parameter / Alokasi CIDR	Keterangan & Tujuan
AWS Region	ap-southeast-1 (Singapore)	Region terdekat dengan dukungan fitur penuh dan latensi rendah.
VPC Name & Block	ansible-mysql-cluster-vpc (10.0.0.0/16)	Ruang alamat IP privat utama untuk seluruh lingkungan infrastruktur.
Internet Gateway (IGW)	ansible-mysql-cluster-igw	Menyediakan jalur keluar/masuk internet utama (dan gratis) untuk VPC.
Public Subnet (Dev)	10.0.1.0/24 (cidrsubnet(var.vpc_cidr, 8, 1))	Subnet publik terpadu tempat berdirinya Bastion Host dan MySQL Nodes.
Route Table Public	0.0.0.0/0 -> Internet Gateway	Mengarahkan seluruh trafik publik dari/ke Bastion Host.

B. Alokasi Resource AWS EC2 Instance & Control Node

Meskipun seluruh *instance* diletakkan pada lokasi jaringan yang sama (*Public Subnet*), pemisahan peran tetap dipertahankan secara ketat melalui konfigurasi kontrol akses firewall.

Parameter Instance	EC2 Bastion Host (Jump Server)	EC2 Managed Nodes (MySQL Cluster)
Lokasi Jaringan	Public Subnet (10.0.1.0/24)	Private Subnet (10.0.2.0/24)
Alokasi IP	IP Publik + IP Privat (10.0.1.x)	IP Publik (Untuk akses IGW) + IP Privat (10.0.1.y)
Sistem Operasi	Ubuntu 22.04 LTS	Ubuntu 22.04 LTS
Instance Type	t2.micro (Kondisional)	t2.micro (var.node_count = 2)
Fungsi Utama	Gerbang masuk SSH ProxyJump	Menjalankan DB MySQL & menerima perintah Ansible

C. Konfigurasi Security Group (Firewall Rules)

Infrastruktur ini menggunakan 2 (dua) *Security Group* terpisah. Inilah kunci utama dari Isolasi Logis. Meskipun *Managed Nodes* memiliki akses ke IGW, koneksi internet dari luar akan dibuang (*dropped*) oleh *Security Group*.

1. Bastion Security Group (bastion_sg)

● Inbound Rules:

Protocol	Port	Source / Asal Trafik	Deskripsi & Tujuan
TCP	22 (SSH)	0.0.0.0/0	Akses SSH publik sebagai akses publik untuk masuk ke <i>Bastion Host</i> dari <i>Control Node</i> Ansible.

● Outbound Rules:

Protocol	Port	Source / Tujuan Trafik	Deskripsi & Tujuan
ALL	ALL	0.0.0.0/0	Mengizinkan lalu lintas keluar untuk meneruskan koneksi ke <i>Managed Nodes</i> .

2. Private Subnet Security Group (private_sg)

● Inbound Rules (Dynamic Block via var.ingress_ports):

Protocol	Port	Source / Asal Trafik	Deskripsi & Tujuan
----------	------	----------------------	--------------------

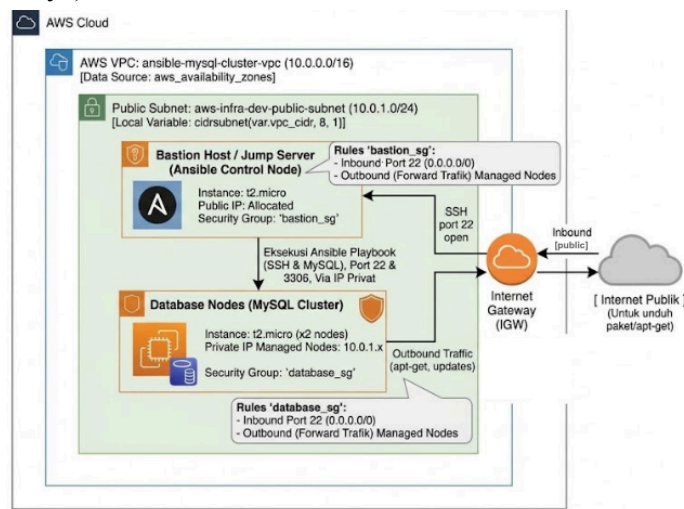
TCP	22 (SSH)	bastion_sg (ID Security Group)	Hanya menerima koneksi SSH yang diteruskan melalui Bastion Host (Ansible Control Node).
TCP	3306 (MySQL)	bastion_sg (ID Security Group)	Hanya menerima koneksi basis data MySQL dari lingkungan terisolasi.

- Outbound Rules:

Protocol	Port	Source / Tujuan Trafik	Deskripsi & Tujuan
ALL	ALL	0.0.0.0/0	Mengizinkan semua lalu lintas keluar (0.0.0.0/0) melalui Internet Gateway agar <i>nodes</i> dapat mengunduh paket (<i>apt-get</i>) secara mandiri tanpa memerlukan NAT Gateway.

1.3 Arsitektur dan Topologi Jaringan Sistem

Berikut adalah alur kerja jaringan hibrida terpadu yang telah dioptimalkan untuk lingkungan *Development* (Hemat Biaya):



Penjelasan Alur Komunikasi Praktis:

1. **Isolasi Keamanan Logis via Firewall:** Seluruh *Managed Nodes* tempat basis data MySQL berada memang berada di dalam *Public Subnet*. Namun, *Security Group* memblokir semua akses masuk (*inbound*) dari internet. Node ini hanya bisa diajak berkomunikasi jika permohonan akses datang dari Bastion Host (*bastion_sg*). Ini menciptakan ruang "privat buatan" yang aman tanpa biaya tambahan jaringan.
2. **Eksekusi Akses Ansible Terpusat dari Bastion Host:** Karena Ansible disiapkan dan dijalankan langsung dari dalam EC2 Bastion Host (sebagai *Control Node*), Ansible mengeksekusi *playbook* langsung ke IP Private milik *Managed Nodes* (10.0.1.x) di dalam jaringan VPC lokal tanpa memerlukan *Virtual Machine* atau terowongan SSH dari komputer lokal.
3. **Jalur Outbound Hemat Biaya (Tanpa NAT Gateway):** Ketika Ansible memberikan perintah instalasi paket (*apt-get update & install mysql-server*), *Managed Nodes* akan melakukan unduhan ke internet. Karena berada di *Public Subnet* dan memiliki IP Publik bawaan EC2, permintaan ini diteruskan langsung ke Internet Gateway (IGW). Hal ini memangkas biaya per jam yang biasanya dihabiskan untuk merawat NAT Gateway.
4. **Pengelolaan State Terpusat dengan S3 & DynamoDB:** Konfigurasi Terraform menggunakan *Remote Backend* di AWS S3 untuk menyimpan berkas *terraform.tfstate* secara terpusat dan terenkripsi. Dipadukan dengan tabel DynamoDB untuk mekanisme *State Locking*, infrastruktur terhindar dari benturan eksekusi (*race condition*).

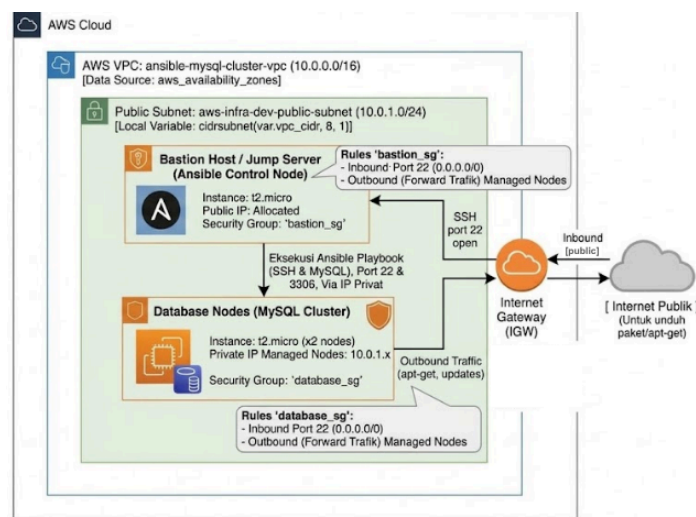
BAB II

2.1 Tahap 1: Provisioning Jaringan & Keamanan (AWS Console)

Metodologi yang diterapkan menggunakan pendekatan *Infrastructure as Code* (IaC) deklaratif berbasis Terraform. Seluruh komponen infrastruktur didefinisikan ke dalam file konfigurasi modular untuk memastikan skalabilitas dan efisiensi biaya. Pada fase *Development* ini, arsitektur dioptimalkan dengan memusatkan seluruh komputasi (EC2) di dalam *Public Subnet* dan meniadakan NAT Gateway, namun tetap mempertahankan isolasi keamanan berlapis melalui konfigurasi *Firewall (Security Group)*.

Fokus pada tahap ini adalah *provisioning* arsitektur jaringan dasar dan komputasi di AWS, sedangkan integrasi konfigurasi *Ansible* akan dilakukan pada tugas tahap selanjutnya.

Topologi Visual Arsitektur Sistem



Implementasi Konsep Advanced Terraform dalam Arsitektur

Konsep Terraform	Lokasi Penerapan & Fungsi
Remote Backend	File backend.tf: Menyimpan <i>state</i> terpusat di AWS S3 dengan <i>State Locking</i> via DynamoDB.
Provider Constraints & Default Tags	File providers.tf: Mengunci versi Terraform & AWS Provider serta menyuntikkan default_tags secara terpusat ke seluruh <i>resource</i> .
Data Sources	File main.tf: Mengambil AMI Ubuntu 22.04 LTS terbaru secara otomatis dan melakukan <i>query Availability Zones</i> aktif.
Type Constraints & Custom Validation	File variables.tf: Membatasi tipe data variabel serta menerapkan aturan validation berbasis fungsi contains() pada variabel environment.
Local Variables & Dynamic Mapping	File locals.tf: Mengalkulasi CIDR Subnet via cidrsubnet(), memetakan instance_types berbasis <i>environment</i> , dan menyusun common_tags.

Built-in Functions (lookup(), file())	File locals.tf & main.tf: Memilih tipe instans sesuai <i>environment</i> via lookup() dan membaca isi <i>public key</i> dari disk lokal via file().
Dynamic Block	File main.tf: Melakukan iterasi aturan Ingress pada Security Group Database Node dari variabel <i>list</i> var.ingress_ports.
Meta-Arguments (count)	File main.tf: Mengontrol jumlah <i>provisioning</i> instans basis data secara dinamis via var.node_count.
Resource Lifecycle Meta-Argument	File main.tf: Menerapkan lifecycle { create_before_destroy = true } pada aws_instance.managed_nodes untuk mencegah <i>downtime</i> saat regenerasi instans.
Conditional Expression (Ternary Operator)	File main.tf & output.tf: Mengontrol pembuatan Bastion Host (var.enable_bastion ? 1 : 0) dan menangani penanganan logika pencetakan IP pada <i>output</i> .
Splat Expression	File output.tf: Mengekstraksi seluruh array IP privat dari kluster instans managed_nodes menggunakan operator [*].

2.2 Analisis dan Penjelasan Kode Berkas Terraform

File konfigurasi di bawah ini telah direvisi secara signifikan. Komponen berbiaya seperti `aws_eip` dan `aws_nat_gateway` telah dihapus. Subnet direduksi menjadi satu lapis (*Public*), namun konsep *Security Group* tetap dipertahankan untuk menjamin keamanan setara *Private Subnet*.

File 1: *providers.tf*

Mendefinisikan AWS Provider dan menyisipkan *default tags* untuk semua *resource* infrastruktur.

```
terraform {
  required_version = ">= 1.5.0" # Provider Constraints: Mengunci minimal versi Terraform
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0" # Provider Constraints: Mengunci versi minor AWS agar terhindar dari breaking changes
    }
  }
}

provider "aws" {
  region = var.aws_region # Provider Configuration: Mengatur region AWS sesuai variabel
  default_tags {
    tags = local.common_tags # Default Tags: Menyuntikkan tag secara otomatis & terpusat ke semua resource
  }
}
```

File 2: *backend.tf*

Mengonfigurasi penyimpanan *Remote State* di S3 agar pengerjaan terhindar dari *race condition*.

```
terraform {
  backend "s3" {
    bucket = "kanjeeng-tfstate-bucket-setup-env-ansible-mysql" # Remote Backend Bucket: Nama bucket S3 untuk menyimpan state
    key    = "aws-infra-dev/terraform.tfstate" # Path to state file in S3 bucket
    region = "ap-southeast-1" # Remote Backend Region: Menentukan region S3 untuk menyimpan state
    dynamodb_table = "terraform-locks" # State Locking: Menggunakan DynamoDB untuk mencegah race condition/bentrok
    encrypt      = true # Enable encryption at rest
  }
}
```

File 3: *variables.tf*

Deklarasi parameter dinamis dan aturan batasannya.

```
variable "aws_region" { # Variable Declaration: Mendefinisikan variabel untuk region AWS
  type      = string # Type Constraints: Membatasi tipe input hanya teks
  default   = "ap-southeast-1"
  description = "Region AWS tempat infrastruktur dibentuk"
}

variable "environment" { # Variable Declaration: Mendefinisikan variabel untuk lingkungan deployment
  type      = string
  default   = "dev"
  description = "Lingkungan deployment (dev, staging, prod)"
  validation { # Custom Validation: Mencegah error akibat salah input nilai
    condition = contains(["dev", "staging", "prod"], var.environment) # Validasi menggunakan fungsi contains()
  }
}
```

```

    error_message = "Nilai environment harus dev, staging, atau prod."
  }
}

variable "vpc_cidr" { # Variable Declaration: Mendefinisikan variabel untuk CIDR VPC
  type      = string
  default   = "10.0.0.0/16"
  description = "Base CIDR Block untuk VPC"
}

variable "node_count" { # Variable Declaration: Mendefinisikan variabel untuk jumlah node database
  type      = number # Type Constraints: Membatasi tipe input hanya angka
  default   = 2
  description = "Jumlah Database Nodes yang dibuat"
}

variable "enable_bastion" { # Variable Declaration: Mendefinisikan variabel untuk mengaktifkan Bastion Host
  type      = bool # Type Constraints: Membatasi input boolean (true/false)
  default   = true
  description = "Flag kondisional untuk mengaktifkan/mematikan Bastion Host"
}

variable "ingress_ports" { # Variable Declaration: Mendefinisikan variabel untuk port yang diizinkan masuk ke Database Security Group
  type      = list(number) # Type Constraints: Membatasi input berupa array/list of numbers
  default   = [22, 3306]
  description = "Port yang diizinkan masuk ke Database Security Group"
}

variable "public_key_path" { # Variable Declaration: Mendefinisikan variabel untuk path public SSH key lokal
  type      = string
  default   = "~/ssh/id_rsa.pub"
  description = "Path lokasi Public SSH Key lokal"
}

```

File 4: *terraform.tfvars*

Menyuplai nilai akhir untuk menyuntikkan ke dalam modul Terraform.

```

# Injeksi: Menyuplai nilai akhir aktual yang akan disuntikkan ke modul Terraform
aws_region    = "ap-southeast-1"
environment   = "dev"
vpc_cidr      = "10.0.0.0/16"
node_count    = 2
enable_bastion = true
ingress_ports = [22, 3306]
public_key_path = "~/ssh/id_rsa.pub"

```

File 5: *locals.tf*

Menyederhanakan alokasi *subnet* hanya menjadi *Public Subnet* untuk menekan biaya (*Cost-Optimized*).

```

locals {
  project_name = "aws-infra-dev"

  common_tags = {
    Project    = local.project_name
    Environment = var.environment
    ManagedBy  = "Terraform"
  }

  # Built-in Function (cidrsubnet(prefix, newbits, netnum)): Mengalkulasi blok CIDR subnet secara otomatis yang menghasilkan: 10.0.1.0/24
  public_subnet_cidr = cidrsubnet(var.vpc_cidr, 8, 1)

  # Dynamic Mapping: Memetakan tipe EC2 berdasarkan environment
  instance_types = {
    dev    = "t2.micro"
    staging = "t3.small"
    prod   = "t3.medium"
  }

  # Built-in Function: Melakukan pencarian data (lookup) dari mapping di atas
  selected_instance_type = lookup(local.instance_types, var.environment, "t2.micro")
}

```

File 6: *main.tf*

NAT Gateway dihilangkan. Seluruh EC2 ditempatkan di *Public Subnet*, namun arsitektur diikat keamanannya melalui `aws_security_group.database_sg` yang memblokir semua akses internet eksternal.

```

# 1. DATA SOURCES
data "aws_ami" "ubuntu" { # Data Sources: Mengambil ID AMI terbaru langsung dari AWS secara otomatis
  most_recent = true
  owners      = ["099720109477"] # Canonical ID
}

```

```

filter {
  name = "name"
  values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
}
filter {
  name = "virtualization-type"
  values = ["hvm"]
}
}

data "aws_availability_zones" "available" { # Data Sources: Melakukan query AZ yang berstatus aktif
  state = "available"
}

# 2. NETWORK ARCHITECTURE (VPC & IGW ONLY)
resource "aws_vpc" "main" { # Resource Declaration: Membuat VPC baru
  cidr_block      = var.vpc_cidr # Resource Declaration: Menggunakan CIDR Block dari variabel yang disuplai
  enable_dns_hostnames = true # Resource Declaration: Mengaktifkan DNS Hostnames untuk EC2 agar bisa diakses via hostname
  enable_dns_support = true # Resource Declaration: Mengaktifkan DNS Support untuk EC2
  tags = { Name = "${local.project_name}-vpc" }
}

resource "aws_internet_gateway" "igw" { # Resource Declaration: Membuat Internet Gateway untuk akses publik
  vpc_id = aws_vpc.main.id
  tags = { Name = "${local.project_name}-igw" }
}

resource "aws_subnet" "public" { # Resource Declaration: Membuat Subnet Publik untuk EC2
  vpc_id      = aws_vpc.main.id
  cidr_block   = local.public_subnet_cidr # Resource Declaration: Menggunakan CIDR Block Subnet Publik yang dihitung secara otomatis dari locals.tf
  availability_zone = data.aws_availability_zones.available.names[0] # Menggunakan AZ pertama yang tersedia
  map_public_ip_on_launch = true # Semua EC2 mendapat IP Publik untuk akses IGW
  tags = { Name = "${local.project_name}-public-subnet" }
}

# 3. ROUTE TABLES
resource "aws_route_table" "public" { # Resource Declaration: Membuat Route Table untuk Subnet Publik
  vpc_id = aws_vpc.main.id
  route { # Resource Declaration: Membuat route default ke IGW untuk akses publik
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }
  tags = { Name = "${local.project_name}-rt" }
}

resource "aws_route_table_association" "public" { # Resource Declaration: Mengasosiasikan Route Table ke Subnet Publik
  subnet_id = aws_subnet.public.id
  route_table_id = aws_route_table.public.id
}

# 4. SECURITY GROUPS (KUNCI ISOLASI LOGIS)
resource "aws_security_group" "bastion_sg" { # Resource Declaration: Membuat Security Group untuk Bastion Host
  name      = "${local.project_name}-bastion-sg"
  description = "Menerima akses SSH dari Publik"
  vpc_id    = aws_vpc.main.id

  ingress { # Resource Declaration: Membuka port SSH dari Publik
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress { # Resource Declaration: Membuka semua akses keluar untuk Bastion Host
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_security_group" "database_sg" { # Resource Declaration: Membuat Security Group untuk Database Nodes
  name      = "${local.project_name}-database-sg"
  description = "Akses ketat hanya dari Bastion Host"
  vpc_id    = aws_vpc.main.id

  dynamic "ingress" { # Dynamic Block: Menghasilkan blok ingress berulang secara dinamis dari list var.ingress_ports
    for_each = var.ingress_ports
    content {
      from_port = ingress.value
      to_port   = ingress.value
      protocol  = "tcp"
      security_groups = [aws_security_group.bastion_sg.id] # Isolasi terbentuk di sini
    }
  }

  egress { # Resource Declaration: Membuka semua akses keluar untuk Database Nodes
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

```

```
# 5. SSH KEY PAIR
resource "aws_key_pair" "deployer" { # Resource Declaration: Membuat SSH Key Pair untuk akses EC2
  key_name   = "${local.project_name}-key"
  public_key = file(var.public_key_path) # Built-in Function: Membaca teks mentah public key dari disk menggunakan file()
}

# 6. EC2 INSTANCES
resource "aws_instance" "bastion" { # Resource Declaration: Membuat Bastion Host EC2
  count          = var.enable_bastion ? 1 : 0 # Conditional Expression (Ternary): Jika 'true' buat 1, jika 'false' lewati pembuatan (0)
  ami            = data.aws_ami.ubuntu.id # Resource Declaration: Menggunakan AMI Ubuntu terbaru dari data source
  instance_type  = local.selected_instance_type # Resource Declaration: Menggunakan instance type yang dihitung dari locals.tf
  subnet_id      = aws_subnet.public.id # Resource Declaration: Menempatkan Bastion Host di Subnet Publik
  vpc_security_group_ids = [aws_security_group.bastion_sg.id] # Resource Declaration: Mengasosiasikan Bastion Host dengan Security Group Bastion
  key_name       = aws_key_pair.deployer.key_name # Resource Declaration: Mengasosiasikan Bastion Host dengan SSH Key Pair yang dibuat sebelumnya

  tags = { Name = "${local.project_name}-bastion" }
}

resource "aws_instance" "managed_nodes" { # Resource Declaration: Membuat Database Nodes EC2
  count          = var.node_count # Meta-Argument: Mengontrol jumlah duplikasi (looping) penciptaan instans EC2
  ami            = data.aws_ami.ubuntu.id # Resource Declaration: Menggunakan AMI Ubuntu terbaru dari data source
  instance_type  = local.selected_instance_type # Resource Declaration: Menggunakan instance type yang dihitung dari locals.tf
  subnet_id      = aws_subnet.public.id # Resource Declaration: Menempatkan Database Nodes di Subnet Publik
  vpc_security_group_ids = [aws_security_group.database_sg.id] # Resource Declaration: Mengasosiasikan Database Nodes dengan Security Group Database
  key_name       = aws_key_pair.deployer.key_name # Resource Declaration: Mengasosiasikan Database Nodes dengan SSH Key Pair yang dibuat sebelumnya

  lifecycle { # Resource Lifecycle: Mencegah downtime; buat mesin baru dulu sebelum hapus yang lama
    create_before_destroy = true
  }

  tags = {
    Name = "${local.project_name}-db-node-${count.index + 1}"
    Role = "Database-Node"
  }
}
```

File 7: *output.tf*

Menampilkan IP yang krusial untuk konfigurasi *Inventory* Ansible di tahap selanjutnya.

```
output "bastion_public_ip" {
  description = "IP Publik Bastion Host untuk akses SSH"
  value       = var.enable_bastion ? aws_instance.bastion[0].public_ip : "Bastion Disabled" # Conditional Expression: Handle string alternatif jika nilai count Bastion 0
}

output "database_nodes_ips" {
  description = "Daftar IP Privat dari Database Nodes (Digunakan untuk SSH ProxyJump)"
  value       = aws_instance.managed_nodes[*].private_ip # Splat Expression: Mengekstrak seluruh private IP dari kumpulan (list) EC2 sekaligus
}

output "vpc_id" {
  description = "ID dari VPC yang terbentuk"
  value       = aws_vpc.main.id
}
```

2.3 Persiapan Lingkungan Kerja & Autentikasi AWS

Sebelum kode Terraform dieksekusi, diperlukan persiapan kredensial akses agar terminal lokal (Windows) memiliki otorisasi untuk melakukan *provisioning* infrastruktur di akun AWS, serta menyiapkan kunci akses SSH yang akan dibaca oleh Terraform.

1. Pembuatan IAM User untuk Terraform (AWS Console)

- Masuk ke layanan **IAM (Identity and Access Management)** di AWS Console.
- Pilih menu **Users** dan klik **Create user** (misal dengan nama: terraform-deployer).
- Pada tahap *Permissions*, pilih **Attach policies directly**.
- Berikan izin akses berikut agar Terraform dapat membangun jaringan, komputasi, dan menyimpan *state* jarak jauh:
 - AmazonEC2FullAccess (Untuk EC2, VPC, Subnet, Security Group).
 - AmazonS3FullAccess (Untuk menyimpan *remote state* terraform.tfstate di S3).
 - AmazonDynamoDBFullAccess (Untuk fitur *State Locking* pada tabel terraform-locks).
- Setelah user dibuat, buat **Access Key** pada tab *Security credentials*, lalu pilih opsi *Command Line Interface (CLI)*. Simpan Access Key ID dan Secret Access Key dengan aman.

User details		
User name terraform-deployer	Console password type None	Require password reset No

Permissions summary			< 1 >
Name ↗	Type	Used as	
AmazonDynamoDBFullAccess	AWS managed	Permissions policy	
AmazonEC2FullAccess	AWS managed	Permissions policy	
AmazonS3FullAccess	AWS managed	Permissions policy	

2. Konfigurasi AWS CLI di Windows (VS Code)

- Buka terminal PowerShell atau Git Bash di dalam Visual Studio Code.
- Jalankan perintah konfigurasi AWS CLI: *aws configure*
- Masukkan kredensial IAM User yang baru saja dibuat:
 - AWS Access Key ID: [Masukkan Access Key Anda]
 - AWS Secret Access Key: [Masukkan Secret Access Key Anda]
 - Default region name: ap-southeast-1 (Sesuai dengan variabel *aws_region* pada konfigurasi).
 - Default output format: json3.

3. Pembuatan SSH Key Pair Lokal

- Di terminal VS Code yang sama, hasilkan kunci SSH untuk disuntikkan ke mesin EC2 nantinya dengan perintah: *ssh-keygen -t rsa -b 4096*
- Tekan *Enter* untuk menyimpan di lokasi bawaan (pada Windows umumnya di C:\Users\NamaUser\.ssh\id_rsa), karena Terraform telah dikonfigurasi untuk membaca dari lintasan ~/.ssh/id_rsa.pub. Biarkan *passphrase* kosong agar otomatisasi tidak terhambat. (Bisa menggunakan key yang sudah ada)

2.4 Eksekusi Infrastructure as Code (Terraform)

Sebelum kita lakukan eksekusi kita akan membuat kedua *resource* ini terlebih dahulu sebelum eksekusi Terraform:

1. Buat S3 Bucket:

- Masuk ke layanan **S3** di AWS Console.
- Klik **Create bucket**.
- Nama Bucket: kanjeeng-tfstate-bucket-setup-env-ansible-mysql
- Region: ap-southeast-1 (Singapura)
- Aktifkan **Bucket Versioning** dan **Encryption** (Sesuai praktik terbaik).

2. Buat DynamoDB Table:

- Masuk ke layanan **DynamoDB** di AWS Console.
- Klik **Create table**.
- Table name: terraform-locks
- Partition key (Primary key): **LockID** (*Wajib persis seperti ini, perhatikan besar kecil hurufnya/String*).

Setelah autentikasi dan kunci SSH siap, eksekusi arsitektur infrastruktur dilakukan melalui alur standar Terraform di dalam terminal VS Code.

1. Inisialisasi (*Init*)

- Jalankan perintah untuk mengunduh modul *provider* AWS versi ~> 5.0 dan menginisialisasi integrasi *backend* S3: terraform init

2. Perencanaan (*Plan*)

- Jalankan perintah validasi untuk meninjau secara simulasi daftar *resource* (VPC, IGW, Subnet, EC2, SG) yang akan dibuat: terraform plan
- Pastikan Terraform berhasil memuat parameter dinamis environment = "dev" dan node_count = 2 tanpa pesan galat (*error*).

3. Penerapan (*Apply*)

- Terapkan konfigurasi untuk mulai membangun infrastruktur di AWS: terraform apply -auto-approve
- Tunggu proses *provisioning* selesai. Pada akhir eksekusi, Terraform akan menampilkan IP Publik dari Bastion Host dan IP Private dari kluster Database Nodes sesuai instruksi pada berkas output.tf. Catat IP Publik Bastion Host tersebut untuk digunakan dalam pengujian akses.

```
Apply complete! Resources: 11 added, 0 changed, 0 destroyed.

Outputs:

bastion_public_ip = "18.143.94.130"
database_nodes_ips = [
  "10.0.1.101",
  "10.0.1.217",
]
vpc_id = "vpc-00288c1a5816740fc"
```

BAB III

VERIFIKASI INFRASTRUKTUR DAN AKSES REMOTE

Berdasarkan hasil eksekusi Terraform yang telah berhasil (*Apply complete!*), infrastruktur Anda kini sudah berjalan di AWS. Berikut adalah draf isi dan panduan langkah demi langkah untuk menyusun BAB 3 di laporan atau dokumentasi Anda.

3.1 Verifikasi Pembuatan Resource di AWS Management Console

Pada bagian ini, Anda perlu mengambil tangkapan layar (screenshot) langsung dari dasbor AWS Management Console sebagai bukti otentik bahwa *resource* telah terbuat.

- **Verifikasi EC2 Instances:** Navigasikan ke menu **EC2 > Instances**.

☐	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...
☐	aws-infra-dev-db-node-1	i-014205080b0d6fe15	Running	t2.micro	2/2 checks passed	ap-southeast-1a	ec2-13-229-63-243.ap-...	13.229.63.243
☐	aws-infra-dev-bastion	i-0a8bb8343f5613512	Running	t2.micro	2/2 checks passed	ap-southeast-1a	ec2-18-143-94-130.ap-...	18.143.94.130
☐	aws-infra-dev-db-node-2	i-0f46af4894a669915	Running	t2.micro	2/2 checks passed	ap-southeast-1a	ec2-13-212-202-236.ap...	13.212.202.236

- **Verifikasi VPC & Subnet:** Navigasikan ke menu **VPC > Your VPCs**. Ambil screenshot VPC bernama aws-infra-dev-vpc (dengan ID vpc-00288c1a5816740fc). Selanjutnya, ke menu **Subnets** aws-infra-dev-public-subnet.

vpc-00288c1a5816740fc / aws-infra-dev-vpc

Details Resource map CIDRs Flow logs Tags Related resources Integrations

Details

VPC ID: vpc-00288c1a5816740fc

DNS resolution: Enabled

Main network ACL: acl-02a49942aa76892bf

IPv6 CIDR (Network border group): -

Encryption control ID: -

State: Available

Tenancy: default

Default VPC: No

Network Address Usage metrics: Disabled

Encryption control mode: -

Block Public Access: Off

DHCP option set: dhcp-00fa5fddaf044e3c8

IPv4 CIDR: 10.0.0.0/16

Route 53 Resolver DNS Firewall rule groups: -

DNS hostnames: Enabled

Main route table: rtb-0308c3b9e43986d7

IPv6 pool: -

Owner ID: 307248929887

subnet-08a0bd2d573677c2b / aws-infra-dev-public-subnet

Details Flow logs Route table Network ACL CIDR reservations Sharing Tags Related resources

Details

Subnet ID: subnet-08a0bd2d573677c2b

IPv4 CIDR: 10.0.1.0/24

Availability Zone: ap-southeast-1a

Network ACL: acl-02a49942aa76892bf

Auto-assign customer-owned IPv4 address: No

IPv6 CIDR reservations: -

Resource name DNS AAAA record: Disabled

Subnet ARN: arn:aws:ec2:ap-southeast-1:307248929887:subnet/subnet-08a0bd2d573677c2b

Available IPv4 addresses: 248

Network border group: sg-southeast-1

Default subnet: No

Customer-owned IPv4 pool: -

IPv6-only: No

DNS64: Disabled

State: Available

IPv6 CIDR: -

VPC: vpc-00288c1a5816740fc | aws-infra-dev-vpc

Auto-assign public IPv4 address: Yes

Outpost ID: -

Hostname type: IP name

Owner: 307248929887

Block Public Access: Off

IPv6 CIDR association ID: -

Route table: rtb-059c7814448a46793 | aws-infra-dev-rt

Auto-assign IPv6 address: No

IPv4 CIDR reservations: -

Resource name DNS A record: Disabled

- **Verifikasi Security Groups:** Navigasikan ke **EC2 > Security Groups**.
 - aws-infra-dev-bastion-sg: Membuka port 22 (SSH) dari 0.0.0.0/0.

sg-016fab803e53e08a2 - aws-infra-dev-bastion-sg

Details Inbound rules Outbound rules Sharing VPC associations Related resources - new Tags

Details

Security group name: aws-infra-dev-bastion-sg

Owner: 307248929887

Security group ID: sg-016fab803e53e08a2

Inbound rules count: 1 Permission entry

Description: Menerima akses SSH dari Publik

Outbound rules count: 1 Permission entry

VPC ID: vpc-00288c1a5816740fc

sg-016fab803e53e08a2 - aws-infra-dev-bastion-sg

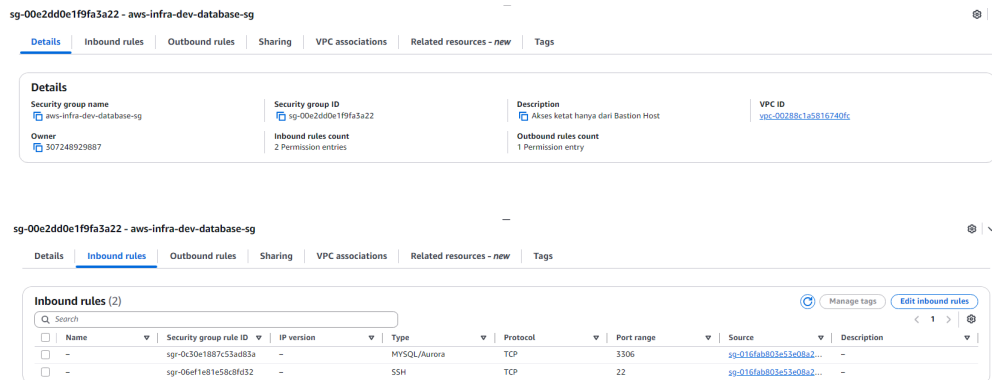
Details Inbound rules Outbound rules Sharing VPC associations Related resources - new Tags

Inbound rules (1)

Q Search

☐	Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
☐	-	sgr-09512e99e89865f4	IPv4	SSH	TCP	22	0.0.0.0/0	-

- aws-infra-dev-database-sg: Membuka port 22 (SSH) dan 3306 (MySQL) yang sumbernya hanya diizinkan dari Security Group Bastion.



3.2 Pengujian Akses Remote (SSH)

Karena arsitektur jaringan menggunakan **Bastion Host**, server Database (Database Node 1 dan Database Node 2) berada di jaringan privat dan tidak memiliki IP Publik. Akses remote ke server Database harus dilakukan dengan melompat (*jumping*) melewati Bastion Host.

Persiapan: Konfigurasi SSH Agent pada Mesin Lokal

Fitur *SSH Agent Forwarding* (parameter *-A*) membutuhkan layanan *ssh-agent* yang aktif di laptop lokal agar otentikasi kunci privat dapat diteruskan secara otomatis tanpa menyimpan file kunci sensitif di dalam Bastion Host.

- Mengaktifkan Service SSH Agent (di Terminal Laptop):**
 - Linux / macOS / Git Bash: `eval "$(ssh-agent -s)"`
 - Windows PowerShell: `Get-Service ssh-agent | Set-Service -StartupType Automatic`
`Start-Service ssh-agent`
- Mengaktifkan Service SSH Agent (di Terminal Laptop):** `ssh-add ~/.ssh/id_rsa`
(Catatan: Sesuaikan `~/.ssh/id_rsa` dengan jalur/nama file kunci .pem milik Anda jika berbeda).
- Verifikasi Kunci Terdaftar:** `ssh-add -l`

Langkah 1: Akses SSH ke Bastion Host

Jalankan perintah berikut di terminal lokal untuk terhubung ke Bastion Host dengan fitur *Agent Forwarding* aktif: `ssh -A ubuntu@18.143.94.130`

```
$ ssh -A ubuntu@18.143.94.130
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-1064-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Tue Sep 29 11:36:40 UTC 2026

System load:  0.0          Processes:    102
Usage of /:   23.9% of 7.57GB Users logged in: 0
Memory usage: 23%         IPv4 address for eth0: 10.0.1.83
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

New release '24.04.5 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Tue Sep 29 11:23:52 2026 from 157.15.63.197
ubuntu@ip-10-0-1-83:~$
```

Langkah 2: Akses SSH ke Database Node 1 (Dari Bastion Host)

Setelah berada di dalam terminal Bastion Host, lakukan pemanggilan SSH langsung ke IP Privat Database Node 1 tanpa perlu membawa file kunci tambahan: `ssh ubuntu@10.0.1.101`

```
ubuntu@ip-10-0-1-83:~$ ssh ubuntu@10.0.1.101
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-1064-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Tue Sep 29 11:52:08 UTC 2026

System load:  0.0               Processes:    102
Usage of /:   24.0% of 7.57GB   Users logged in: 0
Memory usage: 21%              IPv4 address for eth0: 10.0.1.101
Swap usage:  0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

New release '24.04.5 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Tue Sep 29 11:42:36 2026 from 10.0.1.83
To run a command as administrator (user "root"), use "sudo <command>"
See "man sudo_root" for details.

ubuntu@ip-10-0-1-101:~$ |
```

Langkah 3: Akses SSH ke Database Node 2 (Dari Bastion Host)

Keluar dari Database Node 1 kembali ke Bastion Host, lalu hubungkan ke IP Privat Database Node 2: `ssh ubuntu@10.0.1.217`

```
ubuntu@ip-10-0-1-83:~$ ssh ubuntu@10.0.1.217
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-1064-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Tue Sep 29 11:53:28 UTC 2026

System load:  0.0               Processes:    101
Usage of /:   23.1% of 7.57GB   Users logged in: 0
Memory usage: 21%              IPv4 address for eth0: 10.0.1.217
Swap usage:  0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-10-0-1-217:~$
```

Metode Alternatif: Akses Langsung Menggunakan SSH ProxyJump (-J)

Selain menggunakan dua tahap SSH, Anda dapat melakukan lompatan otomatis (*one-liner jump*) langsung dari terminal laptop lokal tanpa perlu mengkonfigurasi ssh-agent:

- Akses ke Database Node 1 via ProxyJump: `ssh -J ubuntu@18.143.94.130 ubuntu@10.0.1.101`
- Akses ke Database Node 2 via ProxyJump: `ssh -J ubuntu@18.143.94.130 ubuntu@10.0.1.217`

BAB IV

KESIMPULAN

4.1 Hasil Implementasi Sistem

Berdasarkan seluruh tahapan perancangan, eksekusi *Infrastructure as Code* (IaC) menggunakan Terraform, hingga pengujian akses *remote*, dapat disimpulkan sebagai berikut:

1. **Otomatisasi Berhasil:** Terraform sukses membangun seluruh infrastruktur AWS secara deklaratif (1 VPC, 1 Public Subnet, IGW, Security Group, SSH Key, serta 3 instance EC2 Ubuntu 22.04 LTS).
2. **Efisiensi Biaya (*Cost-Optimized*):** Arsitektur sukses menekan biaya operasional dengan meniadakan NAT Gateway serta Elastic IP, menggunakan Public Subnet tunggal yang dipadukan dengan IGW murni untuk jalur *outbound*.
3. **Isolasi Logis yang Efektif:** Keamanan basis data tetap terjamin meskipun berada di Public Subnet, berkat pembatasan ketat pada level *firewall* (Security Group) yang hanya mengizinkan akses dari Bastion Host.
4. **Pengelolaan State Aman:** Berkas terraform.tfstate terlindungi dari risiko kerusakan maupun *race condition* menggunakan Remote Backend S3 dan *State Locking* DynamoDB.
5. **Akses Remote Terverifikasi:** Koneksi *remote* ke database privat berhasil dilakukan dengan lancar menggunakan mekanisme *SSH Agent Forwarding* (*ssh -A*) maupun *ProxyJump* (*ssh -J*).

4.2 Saran dan Rekomendasi Arsitektur

Untuk meningkatkan skala sistem dari lingkungan *Development* menuju kelas *Production*, direkomendasikan penyesuaian berikut:

1. **Isolasi Fisik Jaringan:** Memindahkan server database ke Private Subnet murni tanpa alokasi IP publik, serta memanfaatkan NAT Gateway atau VPC Endpoints untuk akses *outbound* yang aman.
2. **High Availability (HA):** Menyebarkan *Database Nodes* ke minimal 2 *Availability Zones* (Multi-AZ) terpisah dan menggunakan layanan *Managed Database* (seperti Amazon RDS).
3. **Keamanan Modern (AWS SSM):** Menggantikan Bastion Host tradisional dan port SSH terbuka dengan *AWS Systems Manager (SSM) Session Manager* demi keamanan akses terminal yang lebih optimal.
4. **Otomatisasi CI/CD:** Mengalihkan eksekusi provisioning dari terminal lokal ke *pipeline* otomatisasi terpusat (seperti GitHub Actions atau GitLab CI).
5. **Audit & Monitoring:** Mengaktifkan *VPC Flow Logs* dan *Amazon CloudWatch* untuk pemantauan performa serta audit keamanan jaringan secara berkala.